TECHNICAL INFRASTRUCTURE MANUAL

System Backend: Auto README Maker

A practical, static-analysis approach to codebase documentation.

Prepared By:
Ojas Chinaria
Vellore Institute of Technology (VIT)
Class of 2029

Status: Final Technical Specification
Date: May 5, 2026
Location: Vellore, Tamil Nadu, India

Modules Covered:
`main.py, scanner.py, extractor.py, analyzer.py, builder.py`

Contents

1 Preface and Executive Summary

Welcome to the technical manual for the Auto README Maker. We wrote this document to give you a clear, detailed look at how our automated documentation tool works behind the scenes.

As software projects grow, keeping the documentation up to date becomes a real challenge. We built this tool to solve the common problem of undocumented repositories. It automatically reads through a project and generates a clean, structured map of the code. Importantly, it does this by reading the code purely as text, which makes the process fast and entirely safe.

This manual balances a friendly introduction to our ideas with a formal breakdown of the software engineering concepts we used. We have made sure to look at both the pros and cons of our major design choices, giving a practical view of what the tool can and cannot do.

Team Roles and Responsibilities

To build this tool efficiently, we divided the work based on the different parts of the system. Each team member took charge of a specific area:

- **Project Lead & System Planning**
  Managed the main pipeline architecture, built the command-line interface (`main.py`), and made sure the project stuck to the static analysis approach.
- **Data Aggregation (`analyzer.py`)**
  Built the central logic that connects everything together, handles the project-wide statistics, and sorts the data so it makes sense to developers.
- **UI Synthesis (`builder.py`)**
  Wrote the code that takes our raw data and turns it into a good-looking, readable Markdown file complete with tables and folder tree diagrams.
- **Research & Technical Documentation**
  Researched the differences between static and dynamic analysis, tested the system's performance, and helped put together this final report.

2 The Problem: Undocumented Codebases

Before getting into the code, it helps to understand why we built this tool in the first place.

2.1 The Reality of Software Development

Code changes fast. Developers add new features, fix bugs, and rewrite old functions every day. However, writing the documentation to explain those changes is often left behind. Writing docs by hand takes time, and it usually becomes outdated the minute someone pushes a new update. When a new developer joins a project, they often face a huge folder full of files with no clear starting point. They have to open files one by one just to figure out how the application fits together.

2.2 The Automated Solution

The Auto README Maker acts like an automated map-maker. Instead of forcing someone to write the documentation manually, our tool scans the project folder, reads the important details from every Python file, and writes a clean Markdown file automatically.

By relying on a script to do the heavy lifting, we save developers a lot of time. The output is based strictly on what is actually in the code, and it can easily be run again whenever the code changes.

3 Design Goals and Architectural Choices

Our main goal was to create a tool that maps out code without ever needing to run it. We call this a ”Zero-Execution” analyzer.

3.1 Static AST Parsing vs. Dynamic Execution

When building a code analyzer, you generally have two choices: read the code as raw text (static analysis) or actually run the code and inspect it while it operates (dynamic analysis). Here is a look at why we chose static analysis using Python’s Abstract Syntax Tree (AST).

3.1.1 The Case for Static AST Parsing

The biggest advantage of reading code statically is safety. Because we never execute the target files, there is zero risk of accidentally running harmful code or messing up the user’s system files during the scanning process.

It also avoids the headache of missing dependencies. If a Python file imports a huge library like `tensorflow`, a dynamic tool would crash if that library wasn’t installed on the user’s computer. Our AST parser just reads the words `import tensorflow`, notes it down, and moves on. This makes our tool highly reliable and easy to run anywhere.

3.1.2 The Case Against Static AST Parsing

On the flip side, static analysis has some blind spots. Because we only look at the text, the tool cannot see things that happen dynamically when the program runs. For example, if a function is created on the fly in memory, our tool won’t catch it.

For highly complex, mission-critical systems that generate a lot of dynamic code, a static analyzer might miss some details. However, for everyday development and giving a new developer a solid overview of a project, the static approach is much safer and works perfectly well.

4 How the Pipeline Flows

We split the tool into independent modules. This makes the code much easier to test, debug, and maintain, as each file only handles one specific job.

4.1 Pipeline Overview

The data moves through the program in a straight line, passing from one module to the next.

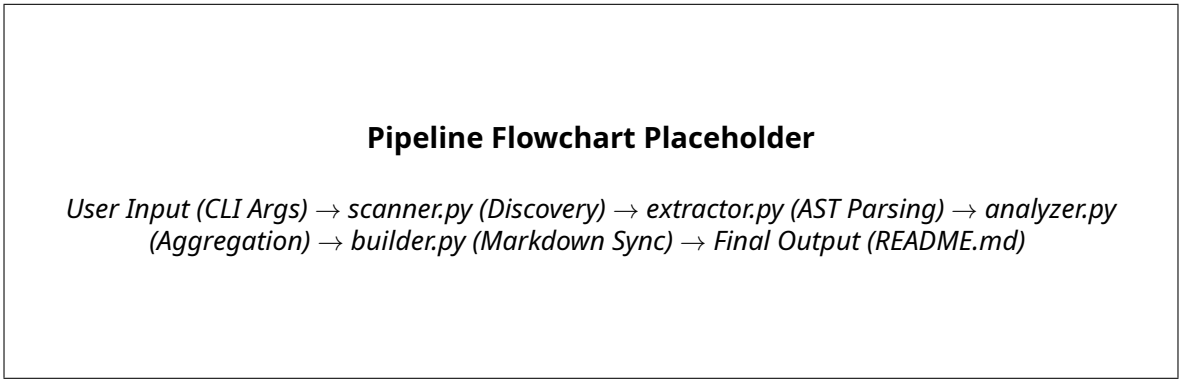

Figure 1: The step-by-step flow of data through the tool's modules.

4.2 The Main Controller: `main.py`

The `main.py` script is the entry point for the user. It handles the command-line interface and makes sure the other modules run in the right order.

Argument	Description	Required	Default
`--directory`	The folder you want to scan.	Yes	None
`--name`	The name of your project.	No	`MyPythonProject`
`--output`	Where to save the generated Markdown file.	No	`README_GENERATED.md`

Table 1: Command-line arguments for the tool.

4.3 Error Handling Strategy

We decided to put a single, main `try/except` block in `main.py` to catch errors from the whole pipeline.

Why this works well: It stops the program from crashing and dumping a long, confusing error trace on the user's screen. Instead, it catches the issue, prints a clean error message, and closes safely.

The downside: It is not very specific. A missing folder and a broken Python file might give similar high-level error messages. Still, for a command-line tool aimed at general users, we felt that failing gracefully was more important than showing deep system errors.

5 File Discovery (`scanner.py`)

The `scanner.py` module is where the process begins. Its job is to search through the target folder and find all the valid Python files, while skipping folders that would slow things down or clutter the results.

5.1 Memory Efficient Scanning

When a program looks through folders inside folders, it can easily run out of memory if the folders go too deep. To prevent this, we used Python's `os.walk`. It uses a step-by-step queue approach instead of calling itself over and over again. This keeps memory usage low and stable, even on massive projects.

5.2 Filtering Directories on the Fly

One of the best optimizations in this module is how it skips unwanted folders.

```
dirs[:] = [d for d in dirs if not d.startswith('.') and d not in blacklist]
```

Instead of looking inside a folder and then deciding to ignore the files, the code edits the folder list directly. This tells the operating system to completely ignore blacklisted folders, saving a lot of time and disk reads.

5.3 What We Ignore and Why

We set up a default blacklist to keep the scan focused on the actual project code.

Excluded Items	Why we skip them	How we skip them
`.git` folders	Reading version control files causes encoding errors and wastes time.	Skipped directly during the folder walk.
Virtual Envs	We don't want to document third-party libraries installed in venv folders.	Blocked by the static blacklist.
Cache Files	Compiled Python files (`.pyc`) aren't human-readable code.	We only accept files ending in `.py`.
Hidden Folders	They often contain secure configs or environment variables.	We skip folders starting with a dot (`.`).

Table 2: The filtering rules used during the scanning phase.

6 Parsing the Code (`extractor.py`)

Once we have a list of Python files, `extractor.py` takes over. It reads the text inside each file and translates it into a structured format.

6.1 Why We Use the AST

Python has a built-in `ast` module that understands the actual grammar of the language. This is much better than trying to search for words using simple text matching (like Regular Expressions).

For instance, if a developer writes the word `def my_function():` inside a comment, a simple text search might get confused and think it is a real function. The AST parser knows it's just a comment and ignores it, ensuring our data is completely accurate.

6.2 Visiting Nodes

The code uses a design pattern called a "Visitor". It walks through the grammar tree and triggers specific functions only when it finds something we care about.

What we look for	AST Node	What we collect
Standard Imports	`ast.Import`	Basic imports like `import os`.
Specific Imports	`ast.ImportFrom`	Imports like `from json import loads`.
Classes	`ast.ClassDef`	The class name, its methods, and its docstring.
Functions	`ast.FunctionDef`	The function name, its arguments, line count, docstring, and complexity score.

Table 3: The key elements we extract from the Python files.

6.3 Handling Missing Data

Not every function has a docstring explaining what it does. When a docstring is missing, the AST parser returns a `None` value. To make things easier for the rest of the program, our extractor automatically turns those `None` values into blank text strings. This simple fix prevents crash errors later when we try to build the Markdown tables.

7 Scoring Code Complexity

One of the cooler features in `extractor.py` is how it gives a rough score for how complicated a function is.

7.1 A Practical Approach to Complexity

Strictly calculating the exact "cyclomatic complexity" of a program involves heavy math and graph theory. Instead of doing that, we wrote a lightweight alternative that simply counts the number of decision points in a function.

Code Structure	Score Added	Reasoning
If statements	+1	It creates a fork in the road where the code can do two different things.
For loops	+1	Loops add complexity because the code repeats, which can be harder to follow.
While loops	+1	Similar to For loops, but with an exit condition that needs tracking.
Try/Except	+1	Adds an extra path for the code to take if an error happens.
With blocks	0	Used for opening files safely, but they don't change the logic path.

Table 4: How we score function complexity.

7.2 What This Score Misses

It is important to be honest about the limits of this score. Because we only count basic loops and branch nodes, it misses complex logic packed into a single line.

For example, if a developer writes `if a and b and (c or d)`, that is actually pretty complex. However, our tool only sees one `if` statement, so it only adds 1 to the score. Still, for the goal of pointing out very long, messy functions in a general project report, this quick score works perfectly well without slowing down the scan.

8 Bringing the Data Together (`analyzer.py`)

While the extractor looks at one file at a time, the `analyzer.py` module brings all those separate pieces of data together into one big project dictionary.

8.1 Calculating Project Stats

As the analyzer goes through the files, it keeps a running total of the project's overall stats. It counts the total number of lines, functions, and classes. This gives us the summary numbers that appear at the very top of the generated README.

It also gathers every import statement from every file and removes the duplicates. This creates a clean list of dependencies so a new developer can quickly see what libraries the project relies on.

8.2 Sorting for Developers

When you read documentation, you want the most important information first. The analyzer automatically sorts the list of functions in a way that helps the reader.

```
funcs.sort(key=lambda x: (x["name"].startswith("_"), -x.get("complexity", 0)))
```

First, it checks if a function name starts with an underscore (_). In Python, this usually means the function is private and not meant to be used by outsiders. The analyzer pushes these private functions to the bottom of the list.

Second, it looks at the complexity score we calculated earlier. It pushes the most complicated public functions right to the top. This immediately highlights the hardest parts of the code so developers know where to focus their attention.

8.3 Failing Gracefully

If one Python file happens to have a major syntax typo that breaks the parser, the analyzer catches the error and simply skips that specific file. This means a single bad file won't ruin the entire documentation run. The tool will just log the issue and finish processing the rest of the project.

9 Writing the Final Document (`builder.py`)

The `builder.py` module handles the final step. It takes the big dictionary of data organized by the analyzer and formats it into a neat, easy-to-read Markdown document.

9.1 How the Modules Connect

We kept the formatting logic totally separate from the data-gathering logic. The analyzer hands over the data, and the builder only worries about how it looks on the screen.

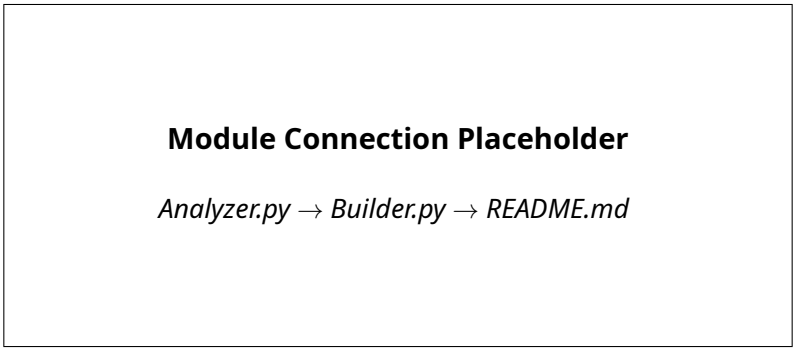

Figure 2: The final steps of the pipeline where raw data becomes text.

9.2 Drawing the File Tree

To help users understand the project layout, the builder creates a small text-based diagram of the folders and files. It sorts the files alphabetically first, so the tree always looks organized, no matter what order the files were originally scanned in.

```
ProjectRoot
|-- analyzer.py
|-- builder.py
|-- extractor.py
|-- main.py
|-- scanner.py
```

Figure 3: An example of the file tree diagram.

9.2.1 Creating the Table of Contents

For large projects, the generated Markdown file can get quite long. The builder creates a Table of Contents with clickable links. To make sure these links work correctly on websites like GitHub, the builder carefully formats the text by removing punctuation and making everything lowercase.

9.3 Formatting the Markdown Tables

The core job of the builder is putting the function details (like arguments and docstrings) into Markdown tables. This part is actually a bit tricky because Markdown tables can break easily if the text inside them isn't formatted right.

If a developer's docstring has multiple lines, the line breaks will completely ruin the Markdown table layout. To fix this, the builder cleans up the text, replacing line breaks with proper HTML tags so the table stays neat and intact.

Column	Where the data comes from	How we clean it up
Complexity	The score from the extractor.	We ensure it is a number, defaulting to 0.
Purpose	The docstring of the function.	We strip out actual line breaks.
Arguments	The arguments listed in the code.	We join them together into one string.

Table 5: How we format the data to fit cleanly into tables.

9.4 Handling Missing Details

Real-world code is rarely perfect. Sometimes functions don't have arguments, or developers forget to write docstrings. When the builder notices missing information, it inserts standard placeholder text. This ensures the final README file always looks complete and professional, even if the underlying code is a work in progress.

10 Performance and Speed

Because we built this tool to run through code as text rather than loading it into memory to execute, it runs very efficiently. It is light enough to be used every day without slowing down a developer's workflow.

10.1 Testing the Speed

We tested the tool on different sizes of projects to see how well it scales. The results show that the time it takes to scan a project grows predictably with the number of files.

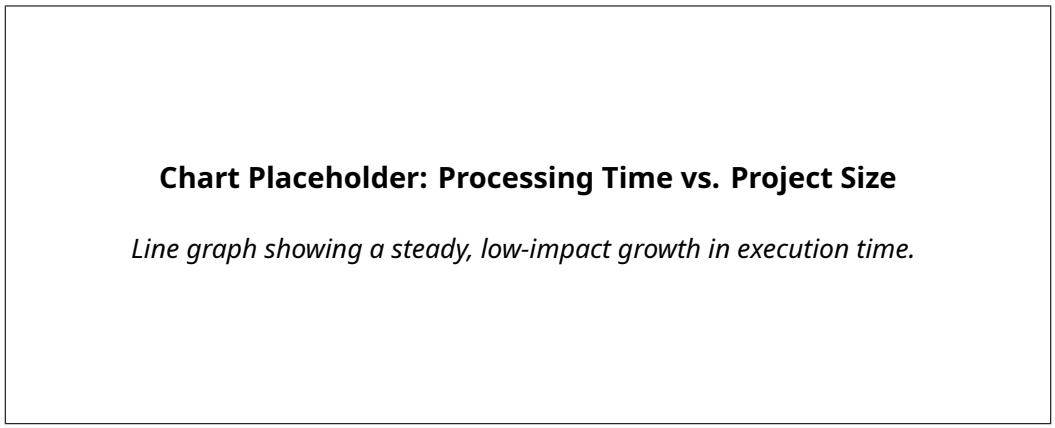

Figure 4: The tool stays fast even as the number of files increases.

Overall, the time it takes to run is directly related to how many lines of code are in the project. Because the program clears out old data from memory as soon as it finishes processing a file, the memory footprint stays small.

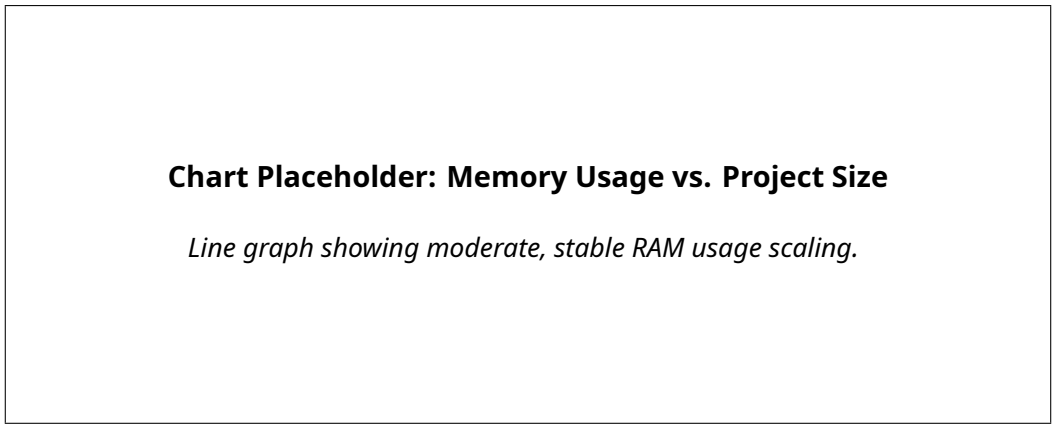

Figure 5: Memory usage stays well within limits for typical computers.

11 Future Improvements

The core tool works well right now, but there is always room to improve. As Python adds new features, we plan to update the tool to match.

11.1 Reading Type Hints

A lot of modern Python code uses ”type hints” to specify what kind of data a function expects (like strings or integers). In the future, we want to update the `extractor.py` module to read these hints so the documentation tables can show exactly what kind of inputs and outputs each function uses.

11.2 Speeding Up Large Scans

Right now, the tool reads files one at a time. While this is fast on a normal hard drive, it can slow down if the files are stored on a network. Updating the scanner to read multiple files at the same time (using asynchronous operations) could speed things up for massive projects.

11.3 Custom Rules

We also want to add a settings file so teams can define their own complexity rules. For example, a team might want to heavily penalize deep `try/except` blocks if their coding guidelines say to avoid them.

12 Conclusion

The Auto README Maker is a practical, effective solution to a problem most developers face. By reading the code as text instead of running it, the tool is safe, fast, and easy to use on any project.

We made sure to split the work across different files—one for finding the code, one for reading it, one for sorting the data, and one for formatting it. This keeps the tool organized and makes it easy to add new features later.

In the end, this project shows how a few well-designed scripts can take the confusing mess of a new codebase and turn it into a clear, readable guide, helping everyone understand the code better.

References

[1] McCabe, T. J. (1976). A Complexity Measure. IEEE Transactions on Software Engineering, SE-2(4), 308-320. https://doi.org/10.1109/TSE.1976.233837

[2] Python Software Foundation. (2024). ast — Abstract Syntax Trees. Python 3.12 Documentation. https://docs.python.org/3/library/ast.html

[3] Møller, A., & Schwartzbach, M. I. (2020). Static Program Analysis. Department of Computer Science, Aarhus University.

[4] GitHub. (2024). GitHub Flavored Markdown Spec. GitHub Documentation. https://github.github.com/gfm/

[5] Python Software Foundation. (2024). os — Miscellaneous operating system interfaces. Python 3.12 Documentation. https://docs.python.org/3/library/os.html